Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 3

Introduction to TensorFlow, PyTorch, JAX, and Keras

The same Dense layer written four times — so the design differences between the frameworks are something you can see rather than something you are told.

Duration 2.5 hours

Instructor Rahman Indra Kesuma, S.Kom., M.Cs. · Prof. Bambang Riyanto Trilaksono

Source Chollet & Watson, Deep Learning with Python 3e — chapter 3

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. Name the **three capabilities** every modern framework provides, and what distinguishes them beyond that.
2. Compute gradients with **GradientTape,** `.backward()`**, and** `jax.grad()`, and explain why the three look so different.
3. Distinguish **stateful imperative** (TF, PyTorch) from **stateless functional** (JAX), and say what that costs when writing a training loop.
4. Switch the Keras 3 backend without changing a line of model code.
5. Write a custom `Layer` with `build()` and `call()`, then drive it through `compile()` and `fit()`.

BOOK
Chapter 3 — full text

NOTEBOOK
01_three_frameworks_side_by_side.ipynb

NOTEBOOK
02_keras3_switch_backend.ipynb

NOTEBOOK
03_custom_layer_and_fit.ipynb

NOTEBOOK
All chapter 3 notebooks

REPO
Author's official notebook

SECTION 3.1

How the field arrived here

Automatic differentiation is from 1964. What was new in 2009 was combining it with GPU computation.

> One number worth keeping: by **mid-2016, more than half** of TensorFlow's users reached it **through Keras**

SECTION 3.1

Three capabilities every one of them has

Automatic differentiation

For any differentiable function you write, not just for a fixed catalogue of layers.

Tensor computation

On CPUs, GPUs, and specialised hardware such as TPUs.

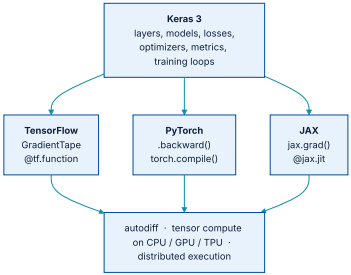
Distributed execution

Across devices and across machines.

Because all of them provide all three, choosing a framework is **not a question of capability**. It is a question of writing style, ecosystem, and speed.

SECTION 3.2

Keras on top, three engines underneath



Keras needs a backend. NumPy can be plugged in but cannot train — it has no gradient API.

SECTION 3.2

Where the line between them falls

> *Keras is like a prefabricated building kit, while TensorFlow, PyTorch, and JAX are like raw materials used in construction.*
>
> Chollet & Watson, section 3.2

Low level — tensors, tensor operations, and backpropagation.

High level — layers combined into models, losses, optimizers, metrics, and training loops.

01

TensorFlow

Immutable tensors, Variables for state, a tape for gradients.

SECTION 3.3

Constants are immutable; state needs a Variable

tensors and variables

PYTHON

```python
import tensorflow as tf

tf.ones(shape=(2, 1))
tf.constant([1.0, 2.0])            # IMMUTABLE — cannot be assigned into

v = tf.Variable(initial_value=tf.random.normal(shape=(3, 1)))
v.assign(tf.ones((3, 1)))          # replace the whole value
v[0, 0].assign(3.0)                # replace part of it
v.assign_add(tf.ones((3, 1)))      # an efficient +=
```

Forgetting this produces a confusing error at the worst moment: **your weights simply refuse to update**

SECTION 3.3

Operations, and one naming detail to notice

tensor operations

PYTHON

```python
a = tf.ones((2, 2))
b = tf.square(a)                    # element-wise
c = tf.sqrt(a)                      # element-wise
d = b + c                           # element-wise
e = tf.matmul(a, b)                 # matrix product
f = tf.concat((a, b), axis=0)       # note: 'axis'

def dense(inputs, W, b):
    return tf.nn.relu(tf.matmul(inputs, W) + b)
```

That `axis` keyword becomes `dim` in PyTorch — a small difference that costs real time when porting code.

SECTION 3.3

GradientTape: record, then ask

gradients PYTHON

```python
input_var = tf.Variable(3.0)
with tf.GradientTape() as tape:
    result = tf.square(input_var)
gradient = tape.gradient(result, input_var)

# A constant is not watched by default — say so explicitly:
c = tf.constant(3.0)
with tf.GradientTape() as tape:
    tape.watch(c)
    result = tf.square(c)
gradient = tape.gradient(result, c)
```

Variables are watched automatically because they are what you normally differentiate. **Constants must be opted in**

SECTION 3.3

Compilation: graph mode and XLA

two levels of compilation

```python
@tf.function
def dense(inputs, W, b):
    return tf.nn.relu(tf.matmul(inputs, W) + b)

@tf.function(jit_compile=True)     # XLA: more aggressive, slower first compile
def dense(inputs, W, b):
    return tf.nn.relu(tf.matmul(inputs, W) + b)
```

This is the same mechanism that later lets a model be **exported without Python** — chapter 6 uses it for deployment.

SECTION 3.3

TensorFlow: where it wins and where it hurts

Strengths

Fast via graph mode and XLA · very feature-complete (string tensors, ragged tensors) · outstanding `tf.data` for preprocessing · **the most mature ecosystem for production, mobile, and browser deployment**.

Weaknesses

A sprawling API with thousands of operations · inconsistent with NumPy in places · less support on Hugging Face than PyTorch.

For teams that must deploy on-premise, that production ecosystem is usually the argument that decides it.

SECTION 3.3

A whole training step in TensorFlow

an end-to-end training step — PYTHON

```python
learning_rate = 0.1

@tf.function(jit_compile=True)
def training_step(inputs, targets, W, b):
    with tf.GradientTape() as tape:
        predictions = model(inputs, W, b)
        loss = mean_squared_error(predictions, targets)
    grad_wrt_W, grad_wrt_b = tape.gradient(loss, [W, b])
    W.assign_sub(grad_wrt_W * learning_rate)      # in-place: W is a Variable
    b.assign_sub(grad_wrt_b * learning_rate)
    return loss

for step in range(40):
    loss = training_step(inputs, targets, W, b)
```

`assign_sub` mutates the variable. Hold that image — **the JAX version of this same loop cannot mutate anything**, and the difference is instructive.

02

PyTorch

Mutable tensors, .backward() fills .grad, eager by default.

SECTION 3.4

Tensors you can assign into

tensors and parameters

PYTHON

```python
import torch                        # the package is 'torch', not 'pytorch'

x = torch.zeros(size=(2, 1))
x[0, 0] = 1.0                       # ASSIGNABLE — unlike TensorFlow

p = torch.nn.parameter.Parameter(data=x)     # marks this as trained state

f = torch.cat((torch.ones((2, 2)), x), dim=0)   # note: 'dim', not 'axis'
```

`Parameter` does not change the maths; it marks a tensor as something an optimizer should own.

No tape — the graph is built as you go

gradients, and the trap PYTHON

```python
input_var = torch.tensor(3.0, requires_grad=True)
result = torch.square(input_var)
result.backward()                    # populates input_var.grad
print(input_var.grad)

input_var.grad = None                # REQUIRED — gradients accumulate otherwise
```

That last line is the classic PyTorch bug. The next `.backward()` **adds** to the existing gradient rather than replacing it, so **training silently stops working**

SECTION 3.4

The three-line incantation

the PyTorch training pattern — PYTHON

```python
class LinearModel(torch.nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.W = torch.nn.Parameter(torch.rand(input_dim, output_dim))
        self.b = torch.nn.Parameter(torch.zeros(output_dim))

    def forward(self, inputs):
        return torch.matmul(inputs, self.W) + self.b

model = LinearModel(2, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

def training_step(inputs, targets):
    loss = mean_squared_error(targets, model(inputs))
    loss.backward()        # 1. compute gradients
    optimizer.step()       # 2. update the weights
    model.zero_grad()      # 3. clear, ready for the next batch
    return loss
```

Learn those three lines in that order and most PyTorch code becomes readable.

SECTION 3.4

PyTorch: where it wins and where it hurts

Strengths

Eager by default, which makes debugging the easiest of the three · **first-class support on Hugging Face**, and that is the single biggest driver of its adoption.

Weaknesses

Internally inconsistent API (`axis` sometimes becomes `dim`) · in the authors' assessment **the slowest of the major frameworks** · `torch.compile()` is still full of edge cases and little used.

If your team already lives on Hugging Face, PyTorch will feel like home. That is a legitimate reason to choose it.

SECTION 3.4

Compilation in PyTorch, and why it is rarely used

torch.compile PYTHON

```python
compiled_model = torch.compile(model)

@torch.compile
def dense(inputs, W, b):
    return torch.nn.relu(torch.matmul(inputs, W) + b)
```

Unlike TensorFlow and JAX, **most PyTorch code runs eagerly, uncompiled**. The book is blunt that the Dynamo compiler is still full of edge cases and that **only a small percentage of users employ it**

03
JAX

Stateless functions. Gradients as a transformation of a function.

SECTION 3.5

Stateless — including the random numbers

arrays, keys, updates PYTHON

```python
import jax
from jax import numpy as jnp

jnp.ones(shape=(2, 1))              # the NumPy API, with no divergence

seed_key = jax.random.key(123)
jax.random.normal(seed_key, shape=(3,))     # same key -> same value, always
key1, key2 = jax.random.split(seed_key)     # how you get a fresh key

x = jnp.array([1, 2, 3], dtype="float32")
new_x = x.at[0].set(10)             # arrays are immutable: you get a new one
```

It reads as extra work, and it is. What you buy is computation that **parallelises automatically without synchronisation**, and results that are exactly reproducible.

SECTION 3.5

jax.grad(): a function in, a function out

the transformation

PYTHON

```python
def compute_loss(state, inputs, targets):
    W, b = state
    predictions = jnp.matmul(inputs, W) + b
    return jnp.mean(jnp.square(targets - predictions))

grad_fn = jax.value_and_grad(compute_loss)      # function -> gradient function
loss, grads = grad_fn((W, b), inputs, targets)  # grads mirrors the shape of state
```

WHAT YOU NEED	WHAT YOU CALL
Gradients only	`jax.grad(f)`
Loss **and** gradients together	`jax.value_and_grad(f)` — cheaper
Plus auxiliary outputs	`jax.value_and_grad(f, has_aux=True)`

SECTION 3.5

What a full training step looks like

a JAX training step — PYTHON

```python
@jax.jit
def training_step(inputs, targets, W, b):
    loss, grads = grad_fn((W, b), inputs, targets)
    grad_W, grad_b = grads
    W = W - grad_W * 0.1
    b = b - grad_b * 0.1
    return loss, W, b                # state comes back out

for step in range(40):
    loss, W, b = training_step(inputs, targets, W, b)
```

> Notice the loop: `W` and `b` are **threaded through by hand**. Nothing is updated in place, anywhere.

SECTION 3.5

JAX: where it wins and where it hurts

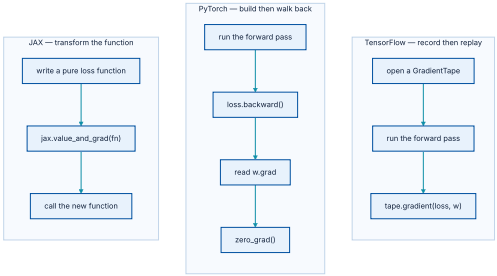
Strengths

In the authors' assessment **the fastest of the three** · perfect NumPy API consistency, so no surprises · built for XLA and TPUs from the beginning.

Weaknesses

Metaprogramming plus compilation makes debugging **markedly harder** than eager execution · low-level training loops are more verbose than TF or PyTorch.

The same job, three ways of asking for it



The gradient is the same quantity in all three; what differs is who holds the state while it is computed.

Side by side

	TENSORFLOW	PYTORCH	JAX
Paradigm	Stateful imperative	Stateful imperative	Stateless functional
Tensors	Immutable (`Variable` for state)	Mutable	Immutable (`.at[].set()`)
Gradients	`GradientTape`	`.backward()` → `.grad`	`jax.grad()` transformation
Compilation	`@tf.function`, XLA	`@torch.compile` (Dynamo)	`@jax.jit` (XLA)
Debugging	Harder in graph mode	Easiest	Hardest (functional + JIT)
Ecosystem	Production tooling	Hugging Face, research	Research, Google scale

About those speed claims

> The book's ranking — JAX fastest, PyTorch slowest, a 20–30% spread and up to 3–5× on large models — is the **authors' assessment**, **not a benchmark this course ran**

Treat it as a direction to investigate, and measure it yourself on your own workload before it becomes a procurement argument.

04

Keras

One model definition, three engines behind it.

SECTION 3.6

Switching backend without touching the model

way 1 — environment variable PYTHON

```python
import os
os.environ["KERAS_BACKEND"] = "jax"

import keras            # MUST come after
print(keras.backend.backend())
```

OUTPUT

```
jax
```

way 2 — ~/.keras/keras.json JSON

```json
{
    "floatx": "float32",
    "epsilon": 1e-07,
    "backend": "tensorflow",
    "image_data_format": "channels_last"
}
```

The ordering is not negotiable: `os.environ[...]` must run **before the first** `import keras`. After Keras is imported, changing it has no effect — and this is the number one confusion in the lab sessions.

Layer: the unit everything is built from

listing 3.22 — a custom Layer PYTHON

```python
import keras

class SimpleDense(keras.Layer):
    def __init__(self, units, activation=None):
        super().__init__()
        self.units = units
        self.activation = activation

    def build(self, input_shape):          # called once, when the first input arrives
        batch_dim, input_dim = input_shape
        self.W = self.add_weight(shape=(input_dim, self.units),
                                 initializer="random_normal")
        self.b = self.add_weight(shape=(self.units,), initializer="zeros")

    def call(self, inputs):
        y = keras.ops.matmul(inputs, self.W) + self.b
        return self.activation(y) if self.activation is not None else y
```

Note that `build()` receives the input shape. The layer does not need to be told it in advance.

SECTION 3.6.2

Automatic shape inference, and what it buys

Weights are created on the first call, not at construction time.

This is why `Sequential` in chapter 2 only had to name the number of units. There was **no `input_shape` anywhere**, and now you know why.

SECTION 3.6.2

Seeing it happen

using the custom layer

PYTHON

```python
my_dense = SimpleDense(units=32, activation=keras.ops.relu)

input_tensor = keras.ops.ones(shape=(2, 784))
output_tensor = my_dense(input_tensor)
print(output_tensor.shape)
```

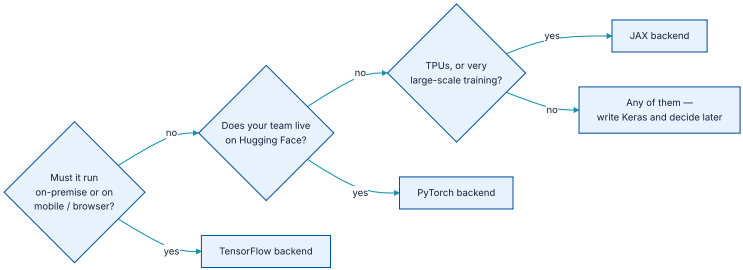
OUTPUT

```
(2, 32)
```

The 784 was never written down. It was read off the data at the moment the layer was first used.

SECTION 3.6

Which backend, in practice



A decision aid, not a law. The point of Keras is that this choice stays reversible.

The authors suggest **JAX** for best performance — but the same Keras code runs on all three, so the decision can be deferred and revisited.

SECTION 3.6.3

Models are graphs of layers — and a choice of hypothesis space

> *The topology of a model defines a hypothesis space. By choosing a network topology, you constrain your space of possibilities to a specific series of tensor operations.*
>
> Chollet & Watson, section 3.6.3

That phrase — **hypothesis space** — is the one from chapter 1. Choosing an architecture is choosing what the model is allowed to learn. Chapter 7 covers the three ways of expressing that choice.

SECTION 3.6.4

compile(): three decisions, spelled out

listing 3.26 — two equivalent forms PYTHON

```python
model.compile(
    optimizer="rmsprop",
    loss="mean_squared_error",
    metrics=["accuracy"],
)

model.compile(
    optimizer=keras.optimizers.RMSprop(learning_rate=1e-4),
    loss=keras.losses.MeanSquaredError(),
    metrics=[keras.metrics.BinaryAccuracy()],
)
```

Built in and ready to use: **SGD, RMSprop, Adam**; losses including the crossentropies and MSE; metrics including accuracy, AUC, precision, and recall.

SECTION 3.6.5

fit(), and why validation data is not optional

listing 3.29 — training with validation PYTHON

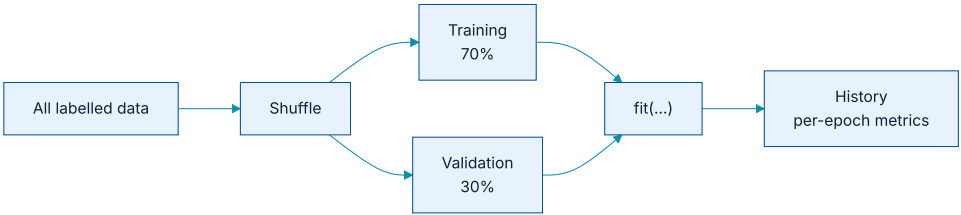
```python
history = model.fit(
    training_inputs, training_targets,
    epochs=5, batch_size=16,
    validation_data=(val_inputs, val_targets),
)
print(history.history.keys())

loss_and_metrics = model.evaluate(val_inputs, val_targets, batch_size=128)
predictions = model.predict(new_inputs, batch_size=128)
```

> *The goal of machine learning is not to obtain models that perform well on the training data … it is to obtain models that perform well in general, particularly on data points that the model has never encountered before.*
>
> Chollet & Watson, section 3.6.5

SECTION 3.6.5

Holding data back, by hand



Shuffle first, then split. Splitting an ordered array is one of the classic ways to get a meaningless validation score.

SECTION 3.6.5

. . . and what that looks like in code

listing 3.28 — a manual split PYTHON

```
indices_permutation = np.random.permutation(len(inputs))
shuffled_inputs = inputs[indices_permutation]
shuffled_targets = targets[indices_permutation]

num_validation_samples = int(0.3 * len(inputs))
val_inputs = shuffled_inputs[:num_validation_samples]
val_targets = shuffled_targets[:num_validation_samples]
training_inputs = shuffled_inputs[num_validation_samples:]
training_targets = shuffled_targets[num_validation_samples:]
```

> Chapter 5 shows what goes wrong when the shuffle is skipped: you can end up training on classes 0–7 and testing on 8–9, and **the code will not complain**

SECTION 3.6.6

Two ways to run inference, and when each is right

inference

PYTHON

```python
predictions = model(new_inputs)                          # all at once
predictions = model.predict(new_inputs, batch_size=128)  # batched, returns NumPy
```

`predict()` iterates in batches, so it **will not exhaust memory** on a large array, and it hands back NumPy rather than backend tensors.

SECTION 3.7

Why this stack is a safe thing to learn

- **Python has won** the ML and data science ecosystem; the authors see no replacement within fifteen years.
- All four frameworks are **stable**. New ones may appear, but are unlikely to displace existing workflows.
- New hardware — AMD GPUs and others — must integrate with the existing frameworks, so it **does not disrupt your code**.

> Which is the actual argument for Keras: it has provided **future-proof stability since 2015**, and a pluggable backend is what lets it keep doing so.

SECTION 3.6.4

Metrics are watched; loss is optimised

> A **metric** is monitored but **never optimised for**. Only the **loss** is minimised. Confusing the two is an expensive mistake, and chapters 5 and 6 return to exactly why.

The short version: many of the things you actually care about — ROC AUC, for instance — cannot be used as a loss because they are not differentiable.

SUMMARY

What has to survive this chapter

1. Every major framework gives you **autodiff, GPU/TPU tensor computation, and distributed execution**. The rest is style.
2. **TensorFlow** — immutable tensors plus `Variable`, `GradientTape`, and the strongest production ecosystem.
3. **PyTorch** — mutable tensors, `.backward()`, eager, king of Hugging Face. Never forget `zero_grad()`.
4. **JAX** — stateless functional, `jax.grad()` as a transformation, explicit random keys, fastest by the authors' account.
5. **Keras 3** sits on all three. Set `KERAS_BACKEND` **before** `import keras`.
6. A `Layer` with `build()` + `call()` is the unit everything is made of, and input shapes are inferred on first use.

NOTEBOOK

01_three_frameworks_side_by_side.ipynb

NEXT

Chapter 4 — Classification and regression